

Module 3.2: Hadoop Architecture and HDFS

Contents

Hadoop I/O	1
Data Integrity	2
Compression.....	2
Serialization	3
Avro	3
File-Based Data Structures	4

Hadoop I/O

Data Input and Output

Hadoop I/O (Input/Output) refers to the mechanisms used by Hadoop to read data from storage, process it, and write the results back to storage. Efficient I/O operations are important for handling large-scale data processing tasks.

Features

- Supports reading and writing huge datasets.
- Works closely with HDFS and MapReduce.
- Provides various input and output formats.

- Optimized for distributed processing.

Importance

- Ensures efficient data access.
- Improves processing performance.
- Supports different data types and file formats.

Input Formats

InputFormat defines how input files are divided into Input Splits and how records are read for processing. **Functions**

- Splits input data into manageable chunks.
- Assigns data to Mapper tasks.
- Converts raw data into key-value pairs.

Types of Input Formats

1) TextInputFormat

- Default input format in Hadoop.
- Each line is treated as a record.
- Key = Byte offset.
- Value = Line content.

2) KeyValueTextInputFormat

- Splits each line into key and value pairs.
- Suitable for structured text data.

3) SequenceFileInputFormat

- Reads data stored in Sequence Files.
- Efficient for binary data.

4) NLineInputFormat

- Processes a fixed number of lines per split.
- Useful for customized data partitioning.

Output Formats

OutputFormat specifies how processed data is written to storage. **Functions**

- Writes MapReduce results.
- Determines output file structure.
- Handles output validation.

Types of Output Formats

1) TextOutputFormat

- Default output format.
- Stores output as text files.
- Produces human-readable results.

2) SequenceFileOutputFormat

- Writes output in binary Sequence File format.
- Suitable for large datasets.

3) MapFileOutputFormat

- Creates indexed output files.
- Supports faster searching.

Advantages

- Flexible storage options.
- Efficient handling of large outputs.
- Supports multiple file formats.

Data Integrity

Data Integrity refers to maintaining the accuracy, consistency, and reliability of data during storage, transfer, and processing. **Importance**

- Prevents data corruption.
- Ensures reliable results.
- Improves system trustworthiness.

Checksum Mechanism

A checksum is a calculated value used to verify that data has not been altered or corrupted. **Working**

1. Data is divided into blocks.
2. Hadoop generates a checksum for each block.
3. Checksums are stored separately.
4. During reading, a new checksum is calculated.
5. Both values are compared.

Benefits

- Detects corruption quickly.
- Ensures data reliability.
- Supports fault tolerance.

Error Detection

Error detection identifies damaged or altered data during storage or transmission. **Methods**

- Checksum verification.
- Block comparison.
- Replica validation.
- 4. Corrupted block is replaced automatically.

Advantages

- Maintains data accuracy.
- Reduces risk of data loss.
- Improves system reliability.

Process

1. Data is read from HDFS.
2. Checksum is verified.
3. If corruption is detected, Hadoop retrieves another replica.

Compression

Compression is the process of reducing the size of data files by encoding information more efficiently. **Need**

- Saves storage space.
- Reduces network traffic.
- Improves data transfer speed.
- Lowers storage costs.
- Enhances processing efficiency.

Types of Compression

- 1. Lossless Compression:** Data can be perfectly reconstructed after decompression. **Examples:** Gzip, Bzip2, Snappy, LZO. **Features**
 - No data loss.
 - Suitable for text and business data.
- 2. Lossy Compression:** Some data is permanently removed during compression. **Examples:** JPEG, MP3, MPEG. **Features**
 - Higher compression ratio.
 - Commonly used for multimedia files.

Common Hadoop Compression Codecs

Codec	Splittable	Speed
Gzip	No	Fast
Bzip2	Yes	Slow
Snappy	No	Very Fast
LZO	Optional	Fast

Advantages

- Reduces storage requirements.
- Improves data transmission speed.
- Minimizes disk I/O operations.
- Enhances Hadoop performance.
- Reduces infrastructure cost.

Serialization

Serialization is the process of converting an object into a stream of bytes so that it can be stored or transmitted across a network.

- Enables data exchange between systems.
- Supports distributed processing.
- Facilitates data storage.
- Data must be transferred efficiently.
- Objects need to be stored persistently.
- Improves processing performance.

Need for Serialization

- Hadoop nodes communicate through networks.

Benefits

- Faster communication.
- Reduced storage requirements.
- Platform-independent data transfer.

Types

1. Java Serialization

- Built into Java.
- Easy to use.
- Relatively slower.

2. Writable Serialization

- Hadoop's native serialization framework.
- Faster than Java Serialization.
- Optimized for MapReduce.

3. Avro Serialization

- Schema-based serialization.
- Supports multiple programming languages.
- Compact and efficient.

4. Protocol Buffers

- Developed by Google.
- High performance.
- Compact binary format.

Avro

Apache Avro is a data serialization system that stores both data and schema together, enabling efficient data exchange between applications. **Components**

- Schema
- Data File
- Serialization Engine

Features

- Schema-based storage.
- Compact binary format.
- Supports schema evolution.
- Language independent.
- Fast serialization and deserialization.
- Suitable for big data applications.

Schema Example

```
{
  "type": "record",
  "name": "Employee",
  "fields": [
    {"name": "id", "type": "int"},
    {"name": "name", "type": "string"}
  ]
}
```

Advantages

- Efficient storage.
- Faster processing.
- Supports multiple languages.
- Easy schema modification.
- Suitable for large datasets.

Applications

- Hadoop ecosystems.
- Data warehousing.
- Data exchange between systems.
- Stream processing.
- Big data analytics.

File-Based Data Structures

File-based data structures provide efficient methods for storing and accessing large datasets in Hadoop.

1) Sequence Files: A Sequence File is a flat binary file that stores data as key-value pairs.

Features

- Binary storage format.
- Highly efficient.
- Supports compression.
- Suitable for large datasets.

- Easy integration with MapReduce.

Applications

- Log storage.
- Intermediate MapReduce output.
- Large-scale data processing.

Advantages

- Faster processing.
- Reduced storage space.

2) Map Files: A Map File is an indexed Sequence File that enables fast data retrieval.

Components

1. Data File
2. Index File

Advantages

- Quick record retrieval.
- Improved query performance.
- Suitable for large datasets.

Features

- Key-value storage.
- Indexed access.
- Faster searches.
- Efficient lookups.

Applications

- Search systems.
- Lookup services.
- Metadata storage.

3) Array Files: An Array File stores values in an indexed array structure.

Features

- Sequential indexing.

- Efficient storage.
- Supports random access.

Advantages

- Fast retrieval using index.
- Easy data organization.
- Efficient processing.

Applications

- Numerical datasets.
- Scientific computing.
- Large collections of records.

4) Set Files: A Set File is a specialized Hadoop file structure used to store unique keys without duplicate entries.

Features

- Stores only unique elements.
- Supports membership testing.
- Built on Map Files.

Advantages

- Eliminates duplicates.
- Fast lookup operations.

- Efficient storage management.

Applications

- Unique record management.
- Duplicate detection systems.
- Membership verification.
- Data validation tasks.